

Lily: System Design, Guidance and Control of an Autonomous Surface Vehicle

Francisco

[TODO: Course name, institution and instructor – fill in]

July 2, 2026

Abstract

This report documents the system architecture, actuation calibration, control, guidance and simulation stack developed for *Lily*, a differential-thrust Autonomous Surface Vehicle (ASV). We describe the vehicle’s physical configuration and state estimation pipeline, the in-water calibration of the propulsion system, the decoupled surge/yaw PID velocity controller and its thruster mixing, an Integral Line-Of-Sight (ILOS) trajectory-following law, the GPS-waypoint mission-management layer, and the Gazebo-based digital twin used for development and validation. Experimental results for velocity tracking and trajectory-following performance are reported in Section 8. [TODO: Add one or two sentences to the abstract once the Results section is filled in with real numbers.]

Contents

1	The System	3
1.1	Vehicle configuration and thruster layout	3
1.2	Compute and communications architecture	3
1.3	Localization: IMU/GPS fusion	4
2	Motor Calibration	5
2.1	Thrust-to-duty-cycle shaping (motor_thrust2RPM)	5
2.2	VESC configuration and FOC commutation	6
2.3	Thrust curve (ERPM → thrust)	6
3	Vehicle Dynamics	7
4	<i>v/w</i> Control: Kinematics and PID Design	7
4.1	Body-frame kinematics	7
4.2	Thruster mixing	8
4.3	Decoupled surge/yaw PID	8
4.4	Relay-feedback PID autotuning	9
4.5	Mode state machine	10
5	Trajectory Following: Integral Line-Of-Sight (ILOS) Guidance	10
5.1	Path-frame errors	10
5.2	Lookahead point and ILOS heading law	11
5.3	Speed law	11

6	Trajectory Generation from Waypoints: Mission Manager	12
6.1	GPS $\rightarrow$ local ENU conversion	12
6.2	Mission finite-state machine	12
6.3	Example missions	13
7	Simulation Environment (lily_gz)	13
7.1	Overview	14
7.2	Environmental plugins: waves and wind	14
7.3	Vehicle model and worlds	14
8	Results	14
8.1	v/w tracking performance	15
8.2	Trajectory-following performance	16
8.3	Discussion	16
	Conclusion	17

1 The System

Lily is a twin-thruster (differential-thrust) unmanned surface vehicle developed for autonomous survey and cooperative deploy/retrieve missions. The vehicle is controlled by a companion computer running ROS 2, bridged to a MAVLink-speaking autopilot (via `mavros`) that handles low-level attitude/position estimation, and interfaces with two brushless thrusters through VESC electronic speed controllers (ESCs) operating in Field-Oriented Control (FOC) mode.

1.1 Vehicle configuration and thruster layout

The vehicle frame (`lily_description/urdf/lily.urdf`) defines the sensor and actuator mounting points relative to `base_link`, summarized in Table 1 and Figure 1.

Table 1: Sensor and actuator mounting offsets relative to `base_link` (URDF, meters).

Frame	Component	x [m]	y [m]	z [m]
<code>left_engine_link</code>	Port thruster	-0.80	+0.33	-0.15
<code>right_engine_link</code>	Starboard thruster	-0.80	-0.33	-0.15
<code>lidar_sensor</code>	3D LiDAR	+1.00	0.00	+0.70
<code>camera_sensor</code>	Monocular camera	+1.20	0.00	+0.70

The two thrusters are mounted symmetrically about the centerline at the stern, giving a thruster (skid-steer) baseline of

$$L = y_{\text{left}} - y_{\text{right}} = 0.33 - (-0.33) = 0.66 \text{ m}, \quad (1)$$

which is exactly the `wheel_base` parameter (0.66 m) used throughout the control stack (Section 4). **[TODO: Confirm overall hull length/beam/draft and dry mass – the URDF only encodes frame offsets, not the hull geometry. The velocity-controller comment in `velocity_controller_real.yaml` notes an estimated mass $\approx 42 \text{ kg}$ and yaw inertia $I_z \approx 26 \text{ kg m}^2$; confirm/replace with measured values.]**

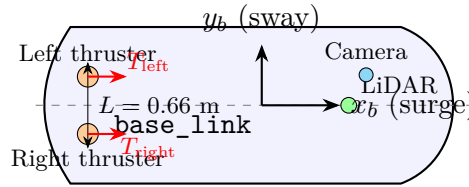


Figure 1: Top-down thruster layout. Two aft-mounted thrusters separated by $L = 0.66 \text{ m}$ produce surge force $F_{\text{surge}} = T_{\text{left}} + T_{\text{right}}$ and yaw moment $M_{\text{yaw}} = (T_{\text{right}} - T_{\text{left}}) \cdot L/2$ about `base_link` (Section 4.2).

1.2 Compute and communications architecture

Figure 2 shows the software architecture. The companion computer runs ROS 2 nodes for guidance, control and perception; `mavros` bridges MAVLink telemetry/commands to/from the autopilot; the autopilot performs attitude and position estimation and exposes RC input, arming and mode services. In simulation, the same topic layout is reproduced by `ros_gz_bridge` against Gazebo (Section 7), so the guidance/control stack is unmodified between sim and field deployment.

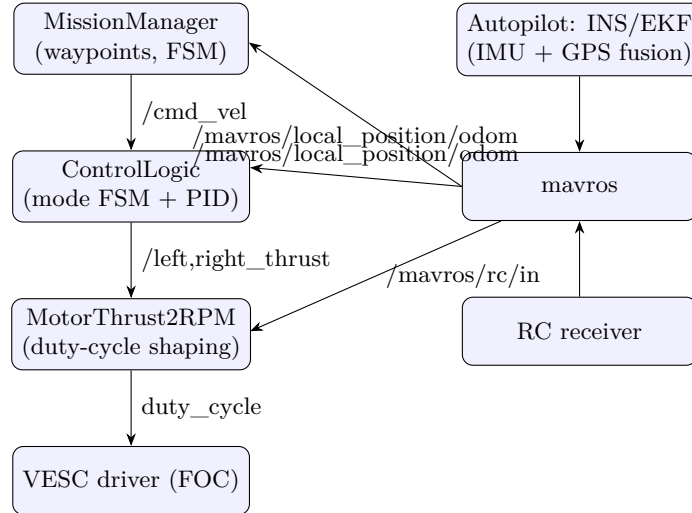


Figure 2: Software architecture. Guidance (MissionManager) and control (ControlLogic) run on the companion computer; the autopilot’s onboard EKF fuses IMU and GPS and exposes a fused odometry estimate through `mavros`. RC input always has top priority in `ControlLogic`.

1.3 Localization: IMU/GPS fusion

Pose and velocity feedback used by every controller in this report (`nav_msgs/Odometry` on `/usv/lily/mavros/local_position/odom`) is a fused estimate produced by the autopilot’s on-board Extended Kalman Filter (EKF), not by a node inside this ROS 2 workspace. The workspace only *consumes* three autopilot outputs bridged by `mavros`:

- `/usv/lily/mavros/imu/data` (`sensor_msgs/Imu`) – angular rate and linear acceleration, high-rate strapdown input;
- `/usv/lily/mavros/global_position/raw/fix` (`sensor_msgs/NavSatFix`) – raw GPS fix;
- `/usv/lily/mavros/local_position/odom` (`nav_msgs/Odometry`) – the EKF’s fused pose/twist output, and the only feedback signal used by `ControlLogic`, `VelocityController` and `MissionManager`.

A local ENU datum is established once per session: in simulation, `gps_origin_publisher.py` parses the `<spherical_coordinates>` tag of the loaded world’s SDF file and republishes it as a `GeoPointStamped` on the same topic name the autopilot uses for its home/origin (`.../global_position/gp_origin`); on the real vehicle this is set by the autopilot at arming. `MissionManager` listens for this once, then converts every mission waypoint from geodetic (lat/lon) to local ENU with a flat-Earth approximation (Section 6.1).

The filter itself runs as autopilot firmware (ArduSub/ArduPilot-class) and is not implemented, tuned, or modified anywhere in this ROS 2 workspace – it is treated as a black box that is simply consumed through the three topics above.

[NOTE: Should we dive into the EKF formulation (state vector, prediction/correction equations), or just describe that we used the ArduSub/ArduPilot on-board EKF as-is, without re-deriving it here?]

[TODO: Confirm which autopilot firmware and EKF (e.g. ArduPilot EKF3, PX4 EKF2) Lily actually runs, and, if we do decide to elaborate, paste in the actual noise/process covariance tuning (EK3_* or EKF2_* parameters) so the description is vehicle-specific rather than generic.]

2 Motor Calibration

[Note for Tiago: Please review this whole motor calibration section – check whether the description of the thrust-shaping, dead-band, ramp and FOC logic below is still accurate and consistent with the current code (`motor_thrust2RPM`, `VESC configs`), and flag anything we should add, correct, or cut before this goes in the report.]

Thrust commands travel through three stages before becoming water flow, as sketched in Figure 3: (i) `ControlLogic/VelocityController` compute a desired thrust in Newtons per side; (ii) `motor_thrust2RPM` converts that thrust command into a bounded, rate-limited ESC duty cycle; (iii) the VESC driver forwards the duty cycle to the ESC, which runs closed-loop FOC commutation on the BLDC thruster motor.



Figure 3: Actuation chain from commanded thrust to delivered thrust.

2.1 Thrust-to-duty-cycle shaping (`motor_thrust2RPM`)

The node first normalizes the requested thrust by the configured `max_thrust`:

$$d_{\text{target}} = \frac{T_{\text{cmd}}}{T_{\text{max}}}, \quad d_{\text{target}} \in [-1, 1]. \quad (2)$$

A dead-band suppresses commands too small to overcome static friction/cogging:

$$d_{\text{target}} = 0 \quad \text{if} \quad |d_{\text{target}}| < d_{\text{base}}, \quad d_{\text{base}} = 0.08. \quad (3)$$

The duty cycle is then rate-limited so the propeller never receives a step input. Three ramp modes are implemented; the deployed configuration uses **mode 2 (quadratic ease-out)**, in which the allowed rate of change grows quadratically with the remaining error, giving a soft start and a snappy finish:

$$e_{\text{ramp}} = \frac{|d_{\text{target}} - d_{\text{last}}|}{d_{\text{max}}}, \quad \text{clamped to } [0, 1] \quad (4)$$

$$\dot{d}_{\text{allowed}} = \underbrace{0.2 \dot{d}_{\text{max}}}_{\text{base rate}} + \underbrace{20 \dot{d}_{\text{max}}}_{\text{explosive rate}} \cdot e_{\text{ramp}}^2, \quad (5)$$

$$\Delta d_{\text{max}} = \dot{d}_{\text{allowed}} \cdot \Delta t, \quad (6)$$

$$d_{\text{last}} \leftarrow d_{\text{last}} + \text{clamp}(d_{\text{target}} - d_{\text{last}}, -\Delta d_{\text{max}}, \Delta d_{\text{max}}), \quad (7)$$

where $\dot{d}_{\text{max}} = \text{max_duty_cycle_rate} = 0.9 \text{ s}^{-1}$. A zero-crossing (sign reversal of the command) triggers an instantaneous stop-before-reverse when `instant_deceleration=true`, modeling the fact that water drag decelerates the hull quickly compared to thruster spin-up; same-direction deceleration is smoothed if `smooth_same_dir_decel=true`. The result is clipped to $d \in [d_{\text{min}}, d_{\text{max}}] = [-0.9, 0.9]$ before being published as the duty-cycle command.

Table 2: `motor_thrust2RPM` parameters (identical for sim and real, `lily_control/cfg/*.yaml`).

Parameter	Value	Meaning
<code>max_duty_cycle / min_duty_cycle</code>	± 0.9	Saturation limits
<code>max_duty_cycle_rate</code>	0.9 s^{-1}	Max slew of duty cycle
<code>ramp_mode</code>	2 (quadratic)	0 none / 1 linear / 2 quadratic ease-out
<code>base_duty_cycle</code>	0.08	Static-friction dead-band
<code>startup_thresh</code>	0.10	Reduced accel. below this target (mode 1)
<code>instant_deceleration</code>	true	Snap to zero on direction reversal
<code>smooth_same_dir_decel</code>	true	Ramp (not snap) when slowing without reversing
<code>loop_rate</code>	50 Hz	Node update rate

2.2 VESC configuration and FOC commutation

The duty-cycle command is sent over a serial link (`transport_drivers/serial_driver`) to a VESC-based ESC running in Field-Oriented Control mode. Table 3 lists the deployed limits (`vesc/vesc_driver/params/vesc_config.yaml`).

Table 3: VESC configuration limits.

Parameter	Value
Serial port	<code>/dev/ttyACM0</code>
Duty cycle range	$[-1.0, 1.0]$ (driver clamps to ± 0.9 upstream)
Motor current limit	$[0, 100] \text{ A}$
Speed limit (ERPM)	± 23250
Brake current range	$[-20000, 200000]$
Servo output range	$[0.15, 0.85]$

FOC commutation requires an in-water motor detection/calibration pass so that winding resistance, inductance and flux-linkage (and, if used, encoder/Hall offset) are identified under realistic load – a submerged propeller presents a very different load curve than the motor spinning in air. **[TODO: Describe here the exact FOC detection procedure you ran in water (VESC “Motor FOC → Detect” wizard values: measured resistance, inductance, flux linkage λ , and any manual current/KV overrides), and the resulting current/torque constants (K_t , K_v) that were applied.]**

2.3 Thrust curve (ERPM $\rightarrow$ thrust)

The relationship between commanded electrical RPM (ERPM) and delivered thrust is modeled with the standard quadratic propeller law (thrust $\propto n^2$ for fixed pitch, fixed advance ratio):

$$T(\text{erpm}) = T_{\max} \left(\frac{\text{erpm}}{\text{erpm}_{\max}} \right)^2, \quad (8)$$

with the reference values used to generate the calibration table (`vesc_driver/scripts/motor_curve.py`): $\text{erpm}_{\max} = 23300$ (matching the VESC `speed_max` ≈ 23250 configured above) and $T_{\max} = 200 \text{ N}$. This curve is what the FOC controller’s closed-loop speed regulation and the thrust $\rightarrow$ duty-cycle node above are calibrated against.

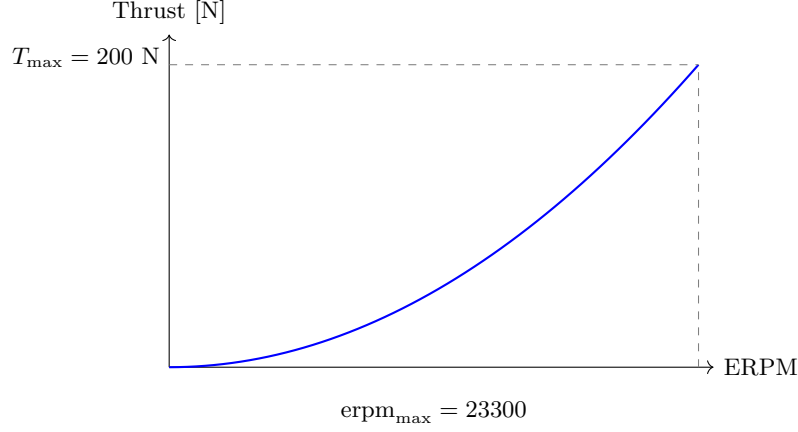


Figure 4: Nominal quadratic thrust curve, Eq. (8), used as the calibration reference.

[TODO: This is the nominal/assumed quadratic model, not a fit to logged bench or in-water force-gauge data. If a real static-thrust-stand or in-water bollard-pull test was performed, insert the measured (erpm, thrust) data points here, refit Eq. (8) (or a more general polynomial) against them, and report the fit residuals.]

3 Vehicle Dynamics

[Note for Diogo: Could you work on this section a bit? The idea is a short, more mechanical/physics-oriented treatment of Lily’s vehicle dynamics (rigid-body equations of motion, added mass, hydrodynamic damping, etc.) to complement the control-oriented surge/yaw model in Section 4. Feel free to pull from the marine/robot-vehicle-dynamics class notes – a standard 3-DOF (surge, sway, yaw) surface-vehicle model (e.g. Fossen-style $M\dot{\nu} + C(\nu)\nu + D(\nu)\nu = \tau$) would fit well here. Doesn’t need to be long, just enough to justify the simplified model used later in the report.]

This section is intended to give a brief, physics-first description of the rigid-body and hydrodynamic effects governing Lily’s motion in the horizontal plane, before Section 4 simplifies this down to the decoupled surge/yaw model actually used by the controller.

[TODO: Placeholder – add: (i) the 3-DOF rigid-body equations of motion in body-fixed coordinates; (ii) added-mass and hydrodynamic damping terms relevant to a planar USV hull; (iii) any simplifying assumptions (e.g. neglecting sway/roll authority) that justify the decoupled surge/yaw model used in Section 4.]

4 v/w Control: Kinematics and PID Design

[Note for Francisco: Please look into this section and add supporting pictures/screenshots (e.g. real thruster mixing test, PID step-response plots from bench/pool tests, autotuner limit-cycle capture) wherever useful.]

4.1 Body-frame kinematics

Lily is modeled as a planar, differential-thrust (skid-steered) surface vehicle. Let v denote surge (forward) speed and w the yaw rate, both expressed in the body frame and measured from the fused odometry (`odom.twist.twist.linear.x`, `odom.twist.twist.angular.z`). Ignoring

sway (no side-thruster authority) the rigid-body equations of motion are

$$m \dot{v} = F_{\text{surge}} - d_v(v) v, \quad (9)$$

$$I_z \dot{w} = M_{\text{yaw}} - d_w(w) w, \quad (10)$$

with $m \approx 42 \text{ kg}$, $I_z \approx 26 \text{ kg m}^2$ (per the controller configuration comments – **TODO: replace with measured mass/inertia**), and $d_v(\cdot)$, $d_w(\cdot)$ lumped (possibly speed-dependent) hydrodynamic drag terms that are not modeled explicitly – instead, their effect is compensated for by the feedback PID controller of Section 4.3 and identified indirectly through relay-based autotuning (Section 4.4). See Section 3 for a more complete treatment of the underlying vehicle dynamics.

4.2 Thruster mixing

The two collinear, aft-mounted thrusters can only produce a surge force and a yaw moment about `base_link` (no direct sway or roll authority):

$$F_{\text{surge}} = T_{\text{left}} + T_{\text{right}}, \quad (11)$$

$$M_{\text{yaw}} = (T_{\text{right}} - T_{\text{left}}) \cdot \frac{L}{2}, \quad L = 0.66 \text{ m (wheel_base)}. \quad (12)$$

Inverting this 2×2 linear mixer gives the thrust allocation actually implemented in `velocity_controller.cpp`:

$$T_{\text{left}} = \frac{1}{2} F_{\text{surge}} - \frac{M_{\text{yaw}}}{L}, \quad (13)$$

$$T_{\text{right}} = \frac{1}{2} F_{\text{surge}} + \frac{M_{\text{yaw}}}{L}. \quad (14)$$

Both are subsequently clamped to $\pm T_{\text{max}}$ (`max_thrust`).

4.3 Decoupled surge/yaw PID

Surge and yaw are controlled by two independent PID loops running at $f_s = \text{loop_rate_hz} = 50 \text{ Hz}$ ($\Delta t = 1/f_s$), closing around the fused-odometry measurements $v_{\text{meas}}, w_{\text{meas}}$ against the commanded $v_{\text{cmd}}, w_{\text{cmd}}$ (from `/cmd_vel`, itself produced either by the ILOS guidance law of Section 5 or an operator teleop command):

$$e_v[k] = v_{\text{cmd}}[k] - v_{\text{meas}}[k], \quad e_w[k] = w_{\text{cmd}}[k] - w_{\text{meas}}[k]. \quad (15)$$

The derivative term is computed *on the measurement* rather than on the error, which avoids the classical “derivative kick” whenever the setpoint changes abruptly:

$$\dot{v}_{\text{meas}}[k] \approx -\frac{v_{\text{meas}}[k] - v_{\text{meas}}[k-1]}{\Delta t}, \quad \dot{w}_{\text{meas}}[k] \approx -\frac{w_{\text{meas}}[k] - w_{\text{meas}}[k-1]}{\Delta t}. \quad (16)$$

The PID outputs (a force and a moment, i.e. the gains carry physical units $\text{N}/(\text{m/s})$ and $\text{N}\cdot\text{m}/(\text{rad/s})$) are

$$F_{\text{surge}}[k] = k_{v,p} e_v[k] + k_{v,i} I_v[k] + k_{v,d} \dot{v}_{\text{meas}}[k], \quad (17)$$

$$M_{\text{yaw}}[k] = k_{w,p} e_w[k] + k_{w,i} I_w[k] + k_{w,d} \dot{w}_{\text{meas}}[k], \quad (18)$$

with integral states updated *after* the tentative output is computed, using *directional* anti-windup: the integral is only allowed to accumulate further in the direction that would deepen an already-saturated output; it may always unwind:

$$I_v[k] = \begin{cases} \text{clamp}(I_v[k-1] + e_v[k]\Delta t, -I_{v,\text{max}}, I_{v,\text{max}}) & \text{if not saturated, or } e_v[k] \cdot I_v[k-1] < 0 \\ I_v[k-1] & \text{otherwise (blocked)} \end{cases} \quad (19)$$

(analogously for I_w), where "saturated" means $|T_{\text{left}}| \geq T_{\text{max}}$ or $|T_{\text{right}}| \geq T_{\text{max}}$ *after* mixing (Eqs. (13)–(14)), and $I_{v,\text{max}} = \mathbf{v_int_max}$, $I_{w,\text{max}} = \mathbf{w_int_max}$. Finally F_{surge} , M_{yaw} are mixed into T_{left} , T_{right} and clamped to $\pm T_{\text{max}}$.

A per-side slew-rate limiter (`SlewRateLimiter`, max rate `max_thrust_rate`) is applied to the final thrust commands *after* mixing, in every control mode (including MANUAL and TELEOP), to protect the driveline from step commands:

$$T_{\text{side}}[k] = T_{\text{side}}[k-1] + \text{clamp}\left(T_{\text{side}}^{\text{cmd}}[k] - T_{\text{side}}[k-1], -\dot{T}_{\text{max}}\Delta t, \dot{T}_{\text{max}}\Delta t\right). \quad (20)$$

Table 4: Decoupled PID gains, integral limits and slew rate (surge v and yaw w).

Config	$k_{v,p}$	$k_{v,i}$	$k_{v,d}$	$I_{v,\text{max}}$	T_{max} [N]	$\dot{T}_{\text{max}}$ [N/s]
Simulation (<code>velocity_controller_sim.yaml</code>)	172.0	41.0	58.0	10.0	100	200
Real vehicle (<code>velocity_controller_real.yaml</code>)	50.0	20.0	40.0	1.0	100	100

Config	$k_{w,p}$	$k_{w,i}$	$k_{w,d}$	$I_{w,\text{max}}$
Simulation	660.0	300.0	40.0	10.0
Real vehicle	50.0	10.0	10.0	1.0

Both configurations share $L = 0.66$ m, $f_s = 50$ Hz, `cmd_timeout` = 0.5 s (command watchdog: `/cmd_vel` is zeroed if stale). **[TODO: The simulation gains were obtained from the adaptive relay autotuner (Section 4.4) and are noticeably higher than the hand-tuned real-vehicle gains – re-run the autotuner on the real boat once in-water access is available and update this table with the resulting K_u , T_u and final gains.]**

4.4 Relay-feedback PID autotuning

Both an offline (`relay_autotuner.py`) and an adaptive online variant (`adaptive_relay_autotuner.py`) implement the classical **Åström–Hägglund relay-feedback method** to identify the plant's ultimate gain and period without needing an explicit dynamic model. The velocity PID is bypassed and a relay of amplitude d drives F_{surge} (surge test) or M_{yaw} (yaw test, with a small constant forward force F_f superimposed so the differential thruster pair retains authority) directly:

$$u(t) = d \cdot \text{sign}(e(t)), \quad e(t) = \text{setpoint} - \text{measurement}, \quad (21)$$

with hysteresis ε on the sign decision to reject chatter. Under relay feedback the plant settles into a limit cycle of period T_u and amplitude a . The describing function of an ideal relay is $N(a) = 4d/(\pi a)$; imposing the Barkhausen criterion $G(j\omega_u)N(a) = -1$ at the limit-cycle frequency $\omega_u = 2\pi/T_u$ gives the ultimate gain and period:

$$K_u = \frac{4d}{\pi a}, \quad T_u = \text{measured oscillation period}. \quad (22)$$

Convergence is declared once the coefficient of variation of amplitude and period over the most recent full cycles drops below a tolerance (5% by default). Three tuning rules are implemented (Table 5); **Tyreus–Luyben** is the default, chosen for its lower overshoot ($\sim 15\%$) on integrating, sluggish, marine-type plants compared to classical Ziegler–Nichols ($\sim 40\%$ overshoot):

$$K_p = K_u/3.2, \quad K_i = \frac{K_p}{2.2 T_u}, \quad K_d = \frac{K_p T_u}{6.3}. \quad (\text{Tyreus–Luyben}) \quad (23)$$

Table 5: Relay-autotuner tuning rules implemented in `relay_autotuner.py`.

Rule	K_p	K_i	K_d
Tyres–Luyben (default)	$K_u/3.2$	$K_p/(2.2 T_u)$	$K_p T_u/6.3$
Ziegler–Nichols (closed loop)	$0.6 K_u$	$K_p/(0.5 T_u)$	$0.125 K_p T_u$
Conservative (PI only)	$K_u/4.5$	$K_p/(2.5 T_u)$	0

The *adaptive* variant additionally re-tunes the relay amplitude d and hysteresis ε online between half-cycles: it increases d if the limit-cycle amplitude is buried in the hysteresis band (weak signal), increases ε if half-cycles are too short (chattering), decreases d on over-excitation, and bumps d upward if no zero-crossing is observed for longer than `stalled_cycle_warn_sec` (stalled/unreachable setpoint) – up to a bounded adaptation budget before aborting.

[TODO: Insert the actual K_u , T_u , limit-cycle amplitude a and resulting gains reported by the autotuner run(s) that produced Table 4’s simulation gains, and (once available) the real-vehicle run.]

4.5 Mode state machine

`ControlLogic` is the sole owner of a 4-state mode machine (Figure 5) and the sole gate to the autopilot’s `SetMode`/arming services; RC input always has final authority. `MissionManager` (Section 6) never asserts a mode directly – it requests `GUIDED`/`POSHOLD` through a service that `ControlLogic` may reject while RC override is active.

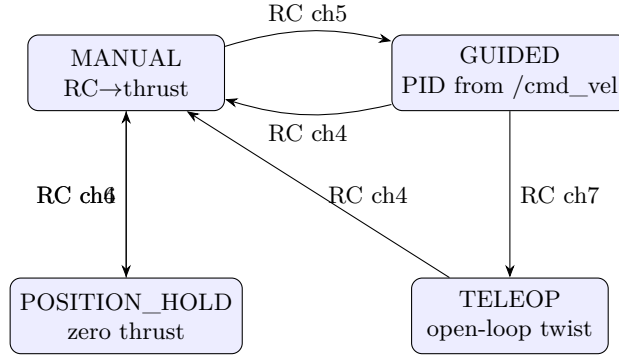


Figure 5: `ControlLogic` mode FSM. Edge-triggered RC channel thresholds ($> 1600 \mu\text{s}$) drive transitions; only a `GUIDED` $\rightarrow$ `MANUAL`/`POSITION_HOLD`/`TELEOP` switch from RC is a hard override that blocks `MissionManager`’s software requests.

5 Trajectory Following: Integral Line-Of-Sight (ILOS) Guidance

[Note for Francisco: Please look into this section and add supporting pictures (e.g. logged path vs. ILOS guidance geometry overlay, real cross-track error traces) wherever useful.]

5.1 Path-frame errors

Given the current path leg from waypoint $\mathbf{p} = (p_x, p_y)$ to the next waypoint $\mathbf{t} = (t_x, t_y)$, the path-tangential angle is

$$\alpha_k = \text{atan2}(t_y - p_y, t_x - p_x). \quad (24)$$

Rotating the vehicle's position $\mathbf{c} = (c_x, c_y)$ relative to $\mathbf{p}$ into the path frame gives the along-track error e_x (used for the lookahead point) and the cross-track error e_y (the signed perpendicular distance to the path, positive to the left):

$$e_x = \cos \alpha_k (c_x - p_x) + \sin \alpha_k (c_y - p_y), \quad (25)$$

$$e_y = -\sin \alpha_k (c_x - p_x) + \cos \alpha_k (c_y - p_y). \quad (26)$$

5.2 Lookahead point and ILOS heading law

A lookahead point is placed a fixed distance Δ (`lookahead_distance`) ahead of the vehicle's along-track projection, clipped so it never overshoots the target waypoint:

$$s = \min(e_x + \Delta, \|\mathbf{t} - \mathbf{p}\|), \quad \ell = \mathbf{p} + s (\cos \alpha_k, \sin \alpha_k). \quad (27)$$

Pure (proportional) LOS would steer the bow toward ℓ using $\psi_d = \alpha_k + \arctan(-e_y/\Delta)$; this drives $e_y \rightarrow 0$ only in the absence of a constant disturbance (current, wind). To reject a constant environmental disturbance with zero steady-state cross-track error, an *integral* term is added on the cross-track error (Integral-LOS, following Fossen & Pettersen):

$$\dot{I}_y = e_y, \quad I_y \leftarrow \text{clamp}(I_y, -I_{y,\max}, I_{y,\max}), \quad (28)$$

$$\tilde{e}_y = e_y + k_i I_y, \quad (29)$$

$$\psi_d = \alpha_k + \arctan\left(\frac{-\tilde{e}_y}{\Delta_{\text{eff}}}\right), \quad \Delta_{\text{eff}} = \max(0.1, \|\ell - \mathbf{c}\|), \quad (30)$$

$$\psi_e = \text{wrap}_{(-\pi, \pi]}(\psi_d - \psi), \quad (31)$$

where $k_i = \text{ki_los}$, $I_{y,\max} = \text{max_integral_e_y}$, and ψ is the vehicle's current yaw. In the deployed mission configurations $k_i = 0$ (Table 6), i.e. the integral action is wired in and available but currently disabled – pure LOS is being used operationally. **[TODO: Decide whether to enable $k_i > 0$ for missions with strong current/wind, and report the cross-track error improvement if you do.]**

5.3 Speed law

Forward speed is commanded proportional to remaining distance, but is weighted down whenever the vehicle is not pointed at the lookahead point – this is what prevents the ASV from surging sideways while turning onto a new leg – and is decelerated inside an arrival radius so that the vehicle settles rather than overshoots each waypoint:

$$d_{\text{tgt}} = \|\mathbf{t} - \mathbf{c}\|, \quad (32)$$

$$\text{align} = \begin{cases} (\max(0, \cos \psi_e))^n & |\psi_e| < \psi_{\text{thr}} \\ 0 & \text{otherwise} \end{cases}, \quad n = \text{align_exponent}, \quad (33)$$

$$v_{\text{allow}} = \begin{cases} \max(0.1, v_{\max} d_{\text{tgt}}/R_{\text{arr}}) & d_{\text{tgt}} < R_{\text{arr}} \\ v_{\max} & \text{otherwise} \end{cases}, \quad (34)$$

$$v_{\text{cmd}} = \text{clamp}(k_{p,\text{lin}} d_{\text{tgt}} \cdot \text{align}, 0, v_{\text{allow}}), \quad (35)$$

$$w_{\text{cmd}} = \text{clamp}(k_{p,\text{ang}} \psi_e, -w_{\max}, w_{\max}). \quad (36)$$

The waypoint is declared reached once $d_{\text{tgt}} < R_{\text{wp}}$ (`waypoint_radius`), at which point the ILOS integrator is reset and the leg advances to the next waypoint pair.

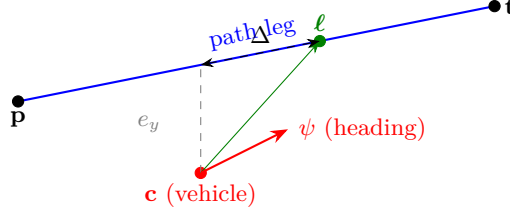


Figure 6: ILOS guidance geometry: cross-track error e_y , lookahead distance Δ , lookahead point ℓ , and desired heading ψ_d toward ℓ (Eq. (30)).

Table 6: ILOS/waypoint guidance parameters (lily_nav mission configs).

Parameter	Square mission	Viana buoys mission	Meaning
$k_{p,\text{lin}}$	0.5	0.5	Linear gain [1/s]
$k_{p,\text{ang}}$	1.5	1.5	Angular gain [1/s]
v_{max}	1.0 m/s	0.5 m/s	Max commanded surge
w_{max}	0.5 rad/s	0.5 rad/s	Max commanded yaw rate
R_{wp} (waypoint_radius)	3.0 m	3.0 m	Waypoint arrival tolerance
R_{arr} (arrival_radius)	5.0 m	5.0 m	Deceleration radius
n (align_exponent)	2.0	2.0	Heading-alignment sharpness
ψ_{thr}	28.6°	28.6°	Max heading err. for forward motion
Δ (lookahead_distance)	3.0 m	3.0 m	ILOS lookahead
k_i (ki_los)	0.0	0.0	Integral gain (disabled)
$I_{y,\text{max}}$	5.0	20.0	Integral clamp
Guidance loop rate	10 Hz	10 Hz	

[TODO: A simpler bearing-only WaypointController (proportional heading law, no lookahead/integral term) also exists in the codebase as a baseline; MissionManager currently instantiates only the ILOS LosController. Mention in the report if the baseline was used for any A/B comparison.]

6 Trajectory Generation from Waypoints: Mission Manager

6.1 GPS $\rightarrow$ local ENU conversion

All guidance math (Section 5) operates in a local, metric East-North-Up (ENU) frame, while missions are authored in geodetic coordinates (latitude/longitude). **MissionManager** converts every waypoint once, on receipt of the local datum (φ_0, λ_0) (Section 1.3), using the same flat-Earth (equirectangular) approximation used throughout the stack (accurate for ranges $\lesssim 10$ km):

$$x = (\lambda - \lambda_0) \cos(\varphi_0) \cdot k, \quad (37)$$

$$y = (\varphi - \varphi_0) \cdot k, \quad k = 111319.5 \text{ m/deg (ArduPilot constant } \approx R_{\oplus} \cdot \pi/180). \quad (38)$$

This gives **MissionManager** a sequence of local ENU waypoints, which is what is actually handed to the ILOS guidance law leg-by-leg (Section 5).

6.2 Mission finite-state machine

MissionManager sequences a deploy/retrieve-style mission as a 6-state FSM, shown in Figure 7, that advances from waypoint to waypoint and reacts to a small set of service calls and arrival

events.

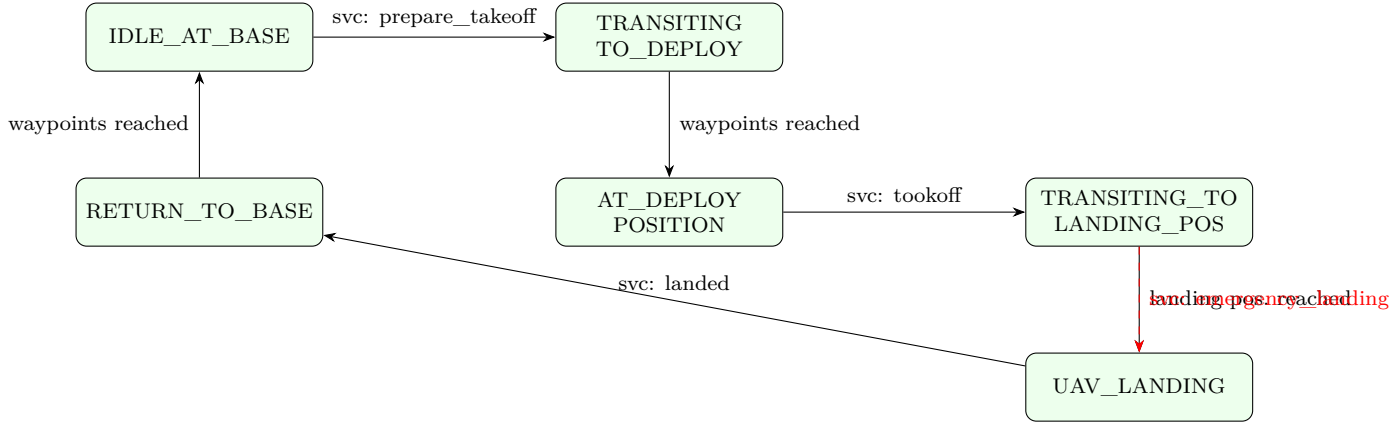


Figure 7: MissionManager finite-state machine: waypoints are consumed sequentially within each `TRANSITING_*` leg, and the FSM advances once the leg’s waypoints are reached or the corresponding service is called.

[**TODO: Confirm/describe the cooperating UAV platform and CONOPS behind the deploy/retrieve service names (`prepare_takeoff`, `tookoff`, `landed`, `emergency_landing`), which indicate this mission profile was built for a cooperative USV→UAV deploy/recovery operation.**]

6.3 Example missions

Table 7 lists the two GPS mission definitions currently checked in, each specified as an ordered sequence of waypoints that `MissionManager` converts to ENU (Section 6.1) and feeds leg-by-leg to the ILOS guidance law.

Table 7: Mission waypoint sets (WGS84 degrees).

Mission	Leg	Waypoints (lat, lon)
Square (FEUP)	Deploy	(41.685117, −8.838916) → (41.685399, −8.839086) → (41.685684, −8.838855)
	Landing	(41.685399, −8.839086)
	Return	(41.685117, −8.838916)
Viana buoys	Deploy	(41.684793, −8.839555) → (41.684922, −8.839550) → (41.685022, −8.839395)
	Deploy pos.	(41.685022, −8.839395)
	Landing	(41.686183, −8.839467)
	Return	reverse of deploy leg

7 Simulation Environment (lily_gz)

[**Note for Francisco: This section please – I’ve trimmed it down a lot: no need to get into ROS 2/bridging/topic-plumbing details here, that’s covered structurally elsewhere. Focus this section on the Gazebo environment itself (worlds, assets) and the wave/wind plugins used to make it a realistic ASV testbed. Please also add pictures/screenshots of the simulated worlds and, if applicable, the wave/wind effects in action.**]

7.1 Overview

The digital twin runs on **Gazebo Harmonic**, reusing several assets originally developed for VRX (the RobotX/VRX surface-vehicle simulation suite): buoys, docks, and land-block obstacles. `lily_environment.launch.py` is config-driven – it takes a `world` launch argument (e.g. `alqueva`, `viana`, `viana_buoys`, `feup`, `empty`) and loads the corresponding `.sdf` world file, along with the models and per-world spawn poses that populate it.

7.2 Environmental plugins: waves and wind

[**TODO: Describe here the Gazebo plugins used to model the water surface (wave field: amplitude/period/direction, whether it produces visual-only waves or also hydrodynamic forcing on the hull) and wind (constant or gusting wind field, force/-torque applied to the hull and any exposed superstructure). Note which worlds/missions had these environmental effects enabled versus running in calm/no-wind conditions, and whether the reported results (Section 8) were obtained with them on or off.**]

7.3 Vehicle model and worlds

The `lily` Gazebo model (`lily_gz/models/lily/model.sdf`) uses dedicated hull/housing/-mount/propeller meshes and PBR textures. Five worlds are checked in: `empty` (unit testing), `feup` (campus pond, used for the square mission), `alqueva` and `viana/viana_buoys` (larger reservoir/coastal environments, the latter with spawned red/dummy buoys for perception-in-the-loop testing), and `dynamic_world` ([**TODO: describe the purpose of `dynamic_world.sdf` – e.g. moving obstacles/waves – if used for the reported results**]).

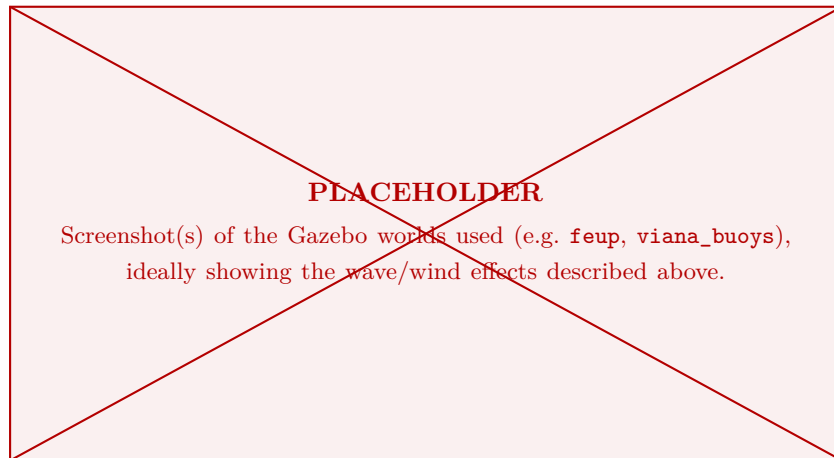


Figure 8: [**TODO: Screenshot(s) of the Gazebo worlds used (e.g. `feup`, `viana_buoys`), ideally showing the wave/wind effects described above.**]

8 Results

[**Note for Francisco: This section is yours too – please fill in the real logs/plots/numbers below once available (from pool/lake/field tests or simulation runs), and add the actual figures in place of the placeholders.**]

This section reports closed-loop performance for (i) the decoupled v/w velocity controller (Section 4) in isolation, and (ii) the full ILOS trajectory-following stack (Section 5–6) on the example missions of Table 7. [**TODO: All figures and numeric values in this section**

are placeholders. Replace each placeholder box with the corresponding plot (e.g. exported from a rosbag via `rqt_plot/plotjuggler/matplotlib`) and fill in the tables once logs from the pool/lake/field tests (or simulation runs) are available.]

8.1 v/w tracking performance

Test protocol. [TODO: Describe exactly how the step tests were run: which mode (GUIDED), commanded step amplitude(s) for v_{cmd} and w_{cmd} , test duration, environment (pool / lake / simulation), and whether surge and yaw were tested independently or jointly.]

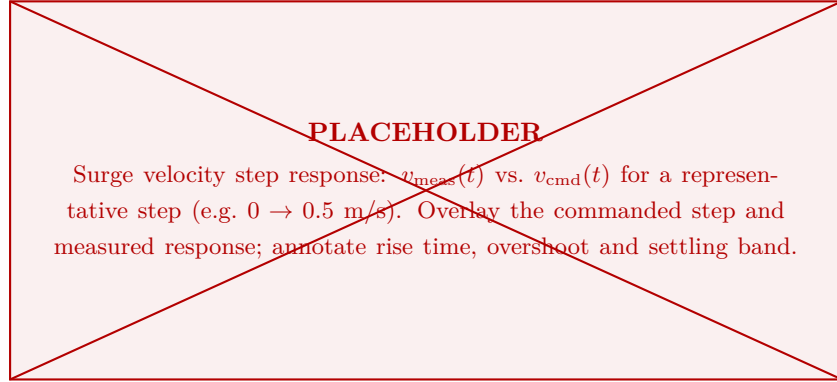


Figure 9: [TODO: Surge velocity step response: $v_{\text{meas}}(t)$ vs. $v_{\text{cmd}}(t)$ for a representative step (e.g. $0 \rightarrow 0.5$ m/s). Overlay the commanded step and measured response; annotate rise time, overshoot and settling band.]

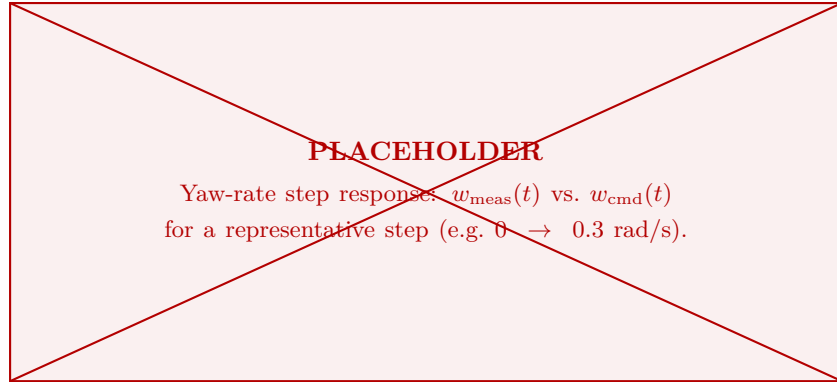


Figure 10: [TODO: Yaw-rate step response: $w_{\text{meas}}(t)$ vs. $w_{\text{cmd}}(t)$ for a representative step (e.g. $0 \rightarrow 0.3$ rad/s).]

Table 8: Velocity-tracking performance metrics (fill in from step-response logs).

Axis	Rise time (10–90%)	Overshoot [%]	Settling time (2%)	Steady-state error
Surge v	TBD	TBD	TBD	TBD
Yaw w	TBD	TBD	TBD	TBD

8.2 Trajectory-following performance

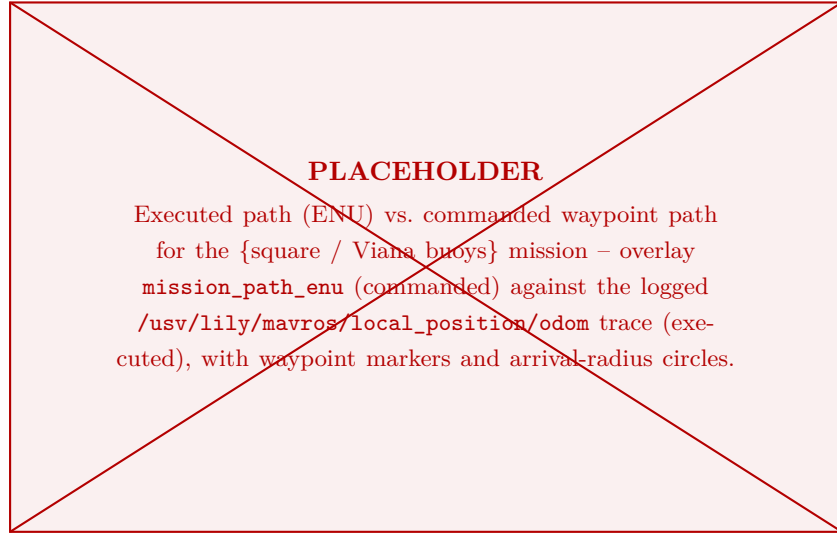


Figure 11: [TODO: Executed path (ENU) vs. commanded waypoint path for the {square / Viana buoys} mission – overlay mission_path_enu (commanded) against the logged /usv/lily/mavros/local_position/odom trace (executed), with waypoint markers and arrival-radius circles.]

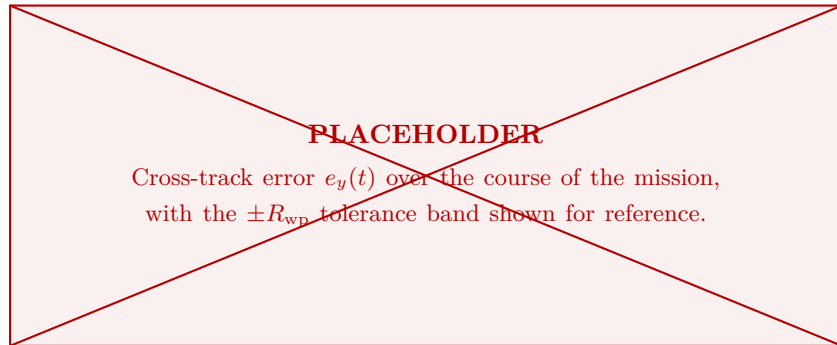


Figure 12: [TODO: Cross-track error $e_y(t)$ over the course of the mission, with the $\pm R_{wp}$ tolerance band shown for reference.]

Table 9: Trajectory-following performance metrics (fill in per mission).

Metric	Square mission	Viana buoys mission
RMS cross-track error e_y	TBD	TBD
Max $ e_y $	TBD	TBD
Mean waypoint arrival error	TBD	TBD
Total mission time	TBD	TBD
Number of RC pause/resume events	TBD	TBD

8.3 Discussion

[TODO: Once the above tables/figures are filled in: (1) comment on whether the simulation-derived PID gains (Table 4) transferred acceptably to the real vehicle

or needed re-tuning; (2) comment on whether $k_i = 0$ (ILOS integral disabled, Table 6) was sufficient given observed cross-track error, or whether enabling the integral term is recommended for future missions; (3) note any observed thruster saturation, RC-override events, or GPS/EKF glitches that affected the results.]

Conclusion

This report described the full autonomy stack of the *Lily* ASV: its physical/sensing configuration and IMU/GPS-fused localization, an in-water-calibrated propulsion chain from commanded thrust to FOC-driven ERPM, a decoupled, relay-autotuned surge/yaw PID velocity controller with thruster mixing, an Integral-LOS trajectory-following law, and a GPS-waypoint mission-management FSM that sequences trajectories from a series of waypoints, together with a Gazebo-based digital twin used for development and validation. **[TODO: Close with 2–3 sentences summarizing the actual quantitative results once Section 8 is filled in.]**